



Continuous Integration, Delivery & Deployment CI/CD

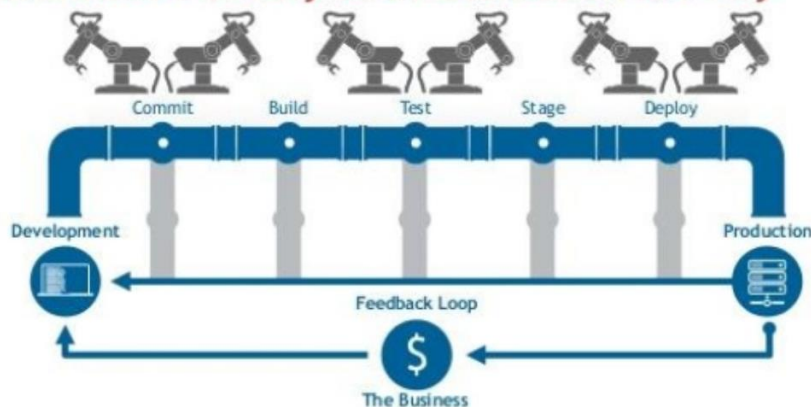
Agenda

- Introduction and Definition
- How it works?
- Benefits of using CI/CD
- Principles
- Tools and Examples
- Future Trends in CI/CD
- Q&A

Anti-patterns of Software Delivery

- Deploy Software Manually
- Deploying to a Production-Like environment (e.g. staging) only after development is complete
- Manually configuration management for Production environment

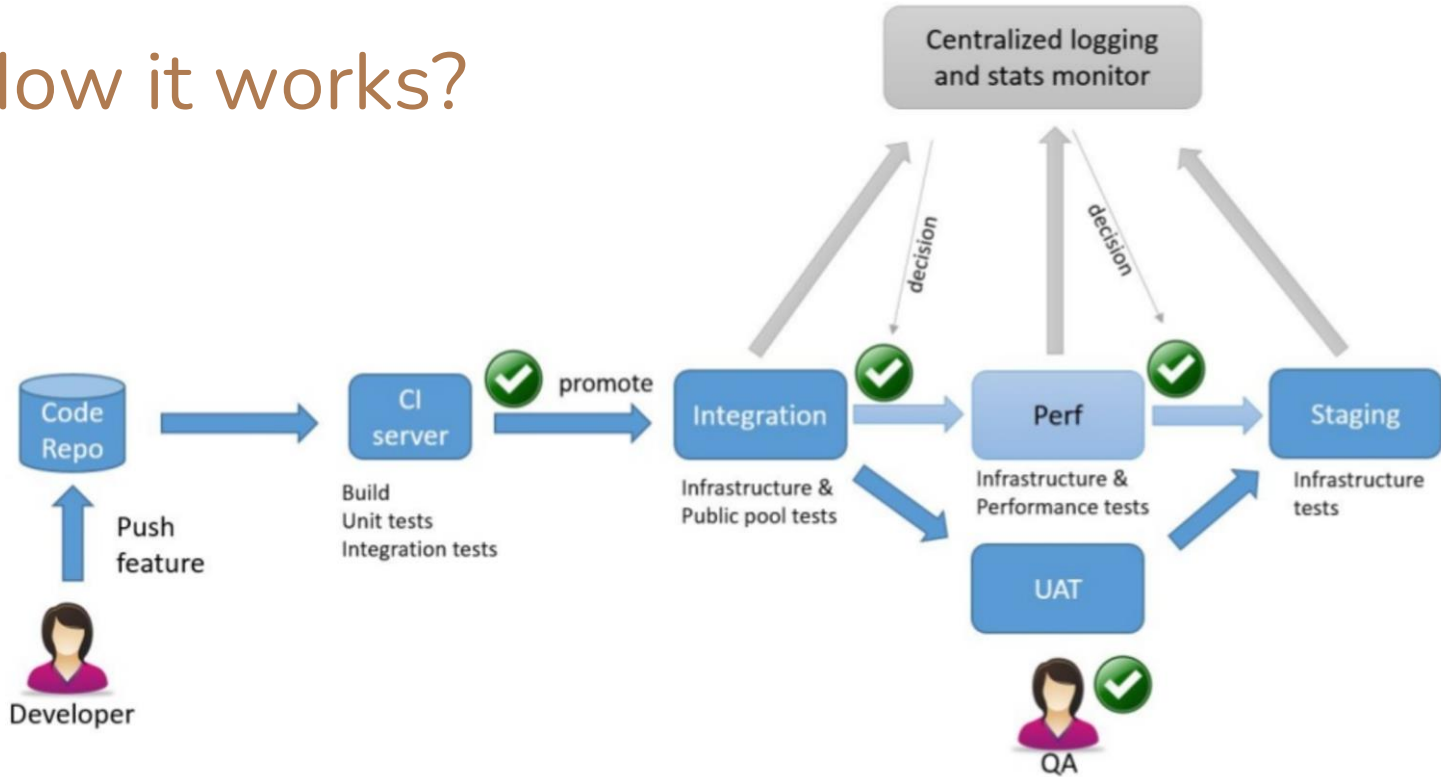
Continuous Delivery and Automation are Key



Definitions

- *Continuous Integration* usually refers to integrating, building, and testing code within the development environment. *Continuous Delivery* builds on this, dealing with the final stages required for production deployment
- *Continuous Delivery* is a software engineering approach in which teams produce software in short cycles, ensuring that the software can be reliably released at any time. It aims at building, testing, and releasing software faster and more frequently - Wiki
- *Continuous Deployment* means that every change goes through the pipeline and automatically gets put into production, resulting in many production deployments every day. In order to do *Continuous Deployment* you must be doing *Continuous Delivery*

How it works?



Benefits of CI/CD

- Reduction of repetitive manual processes → automate everything
- Reducing errors (e.g. human mistakes)
- Lowering stress: fast and easy to deploy / rollback
- Bugs and problems discovered closer to the point in time that they were introduced → improve productivity and quality
- System is deployable more frequently and flexibility, enabling more users feedback → reliable releases
- Improve customer satisfaction

Challenges without CI/CD

- **Manual Processes:** Prone to errors and time-consuming.
- **Inconsistent Environments:** Variations lead to bugs and deployment issues.
- **Integration Issues:** Difficulty merging changes, causing conflicts.
- **Slow Feedback Loop:** Delays in identifying and fixing issues.
- **Lower Release Frequency:** Infrequent updates limit responsiveness.
- **Quality Assurance:** Manual testing can miss defects.
- **Scalability Issues:** Hard to maintain consistency across large teams and projects.

CI/CD in the context of Agile Development

- CI/CD automates the integration and deployment processes, aligning closely with Agile's emphasis on iterative development, continuous improvement, and rapid feedback loops.
- Frequent integration and automated deployments ensure that new features and fixes are processed quickly, enhancing Agile cycles.
- Automated testing in CI/CD pipelines provides immediate feedback on each change, crucial for Agile's adaptive planning and evolutionary development.
- CI/CD facilitates better collaboration among developers, testers, and operations teams by maintaining a single source of truth and automating the flow of changes.
- CI/CD scales the Agile framework to larger, more complex projects by standardizing development, testing, and deployment processes.

CI/CD Tool Options



Jenkins



 **Bamboo**

 **go**
Continuous Delivery



Travis CI



circleci



GitLab



CODESHIP

 powered by
amazon
web services™

Visual Studio Team Services



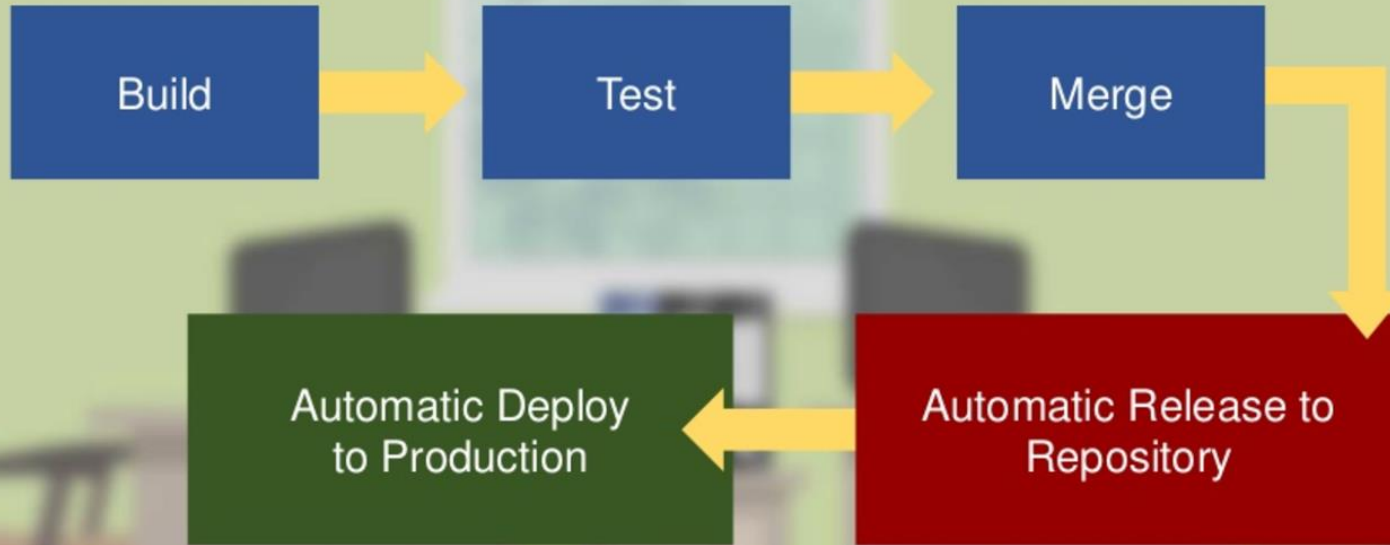
Example of CI/CD – Verizon Prepaid

- **Project overview:** Verizon is a web application that hosted AWS to support interface for processing end-customer transactions through the workflow to activate Verizon Prepaid accounts
 - Tech Stack: Node.js, Angular.js, Docker, AWS
 - Team: 4 DEV, 1 Automation Developer, 1 QA
- **Development Workflow**
 - Two long running branches master and develop with a transition number of feature branches
 - Using GIT Flow
 - Workflow
 - Create feature branch from develop
 - Commits changes to the feature branch
 - Create a Pull Request (PR) targeting the develop branch
 - Merged PR after it passed CI tests and team review
 - Delete feature branch after integration is complete



Principles

CONTINUOUS INTEGRATION

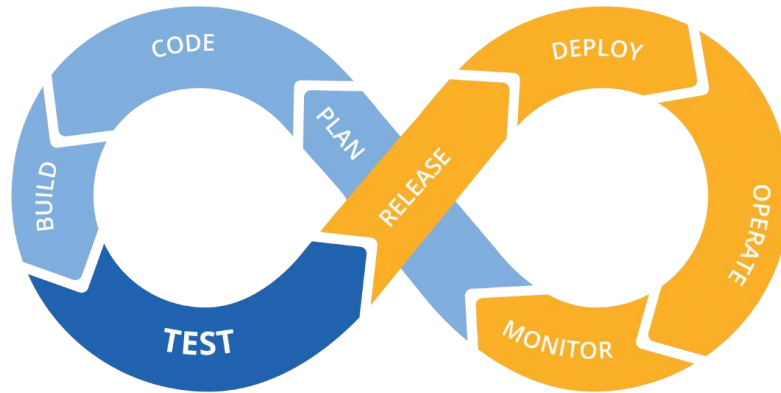


CONTINUOUS
DEPLOYMENT

CONTINUOUS
DELIVERY

What is Continuous Integration?

Continuous Integration (CI) is a DevOps software development practise, that enables the developers to merge their code changes in the central repository, so that automated builds and tests can be run



What is Continuous Integration?

Continuous Delivery is now being valued by all size of organizations, and they are willing to invest in it due to the following reasons:

1

**Avoids merge
conflicts**

2

**Decrease code
review**

3

**Speeds up
development
processes**

4

**Reduces
project backlog**

What is Continuous Delivery?

Continuous Delivery (CI) is a DevOps practice that refers to the building, testing and delivering improvements to the software code. The most important part of CD is that the code is always in a deployable state



What is Continuous Delivery?

Continuous Delivery is now being valued by all size of organizations, and they are willing to invest in it due to the following reasons:

1

**Higher quality
products**

2

**Quicker and
less risky
releases**

3

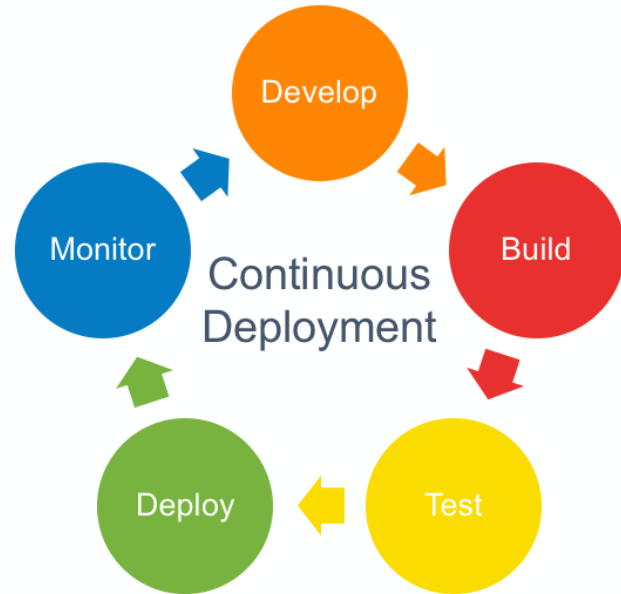
**Customers are
more satisfied**

4

**More efficiency and
cost effectiveness of
the team**

What is Continuous Deployment?

Continuous Deployment (CD) refers to the final stage in the pipeline that refers to the automatic releasing of any developer changes from the repository to the production.



What is Continuous Deployment?

Continuous Delivery is now being valued by all size of organizations, and they are willing to invest in it due to the following reasons:

1

**Faster
development
of products**

2

**Less risky releases
and easy to fix
problems**

3

**Continuous
improvement
in quality**

Difference Between Delivery and Deployment

Continuous Delivery:

- **Goal:** Automate the software release process to ensure that any version can be deployed to production at any time.
- **Process:** Code changes are automatically built, tested, and packaged for release, but deployment is typically a manual step.
- **Focus:** Ensures every change is release-ready, increasing confidence in the quality of new releases.

Continuous Deployment:

- **Goal:** Automate the deployment process to reduce the time elapsed between writing a line of code and using it in production.
- **Process:** Extends continuous delivery by automatically deploying all changes that pass the test phase to production.
- **Focus:** Accelerates the overall release process, enabling immediate user feedback and continuous improvement.

Strategies for Safe Deployment

- **Canary Releases:** Gradually roll out changes to a small subset of users to monitor performance and catch issues before a full rollout.
- **Blue/Green Deployments:** Deploy the new version alongside the old version. Switch traffic either gradually or instantly, based on performance.
- **A/B Testing Deployment:** Test two versions simultaneously to compare performance, usability, and other metrics directly before deciding on a wider release.
- **Ramped Deployment (Rolling Update):** Gradually replace instances of the old version of the application with the new version across servers, reducing risk by spreading it over time.
- **Shadow Deployment:** Deploy the new version alongside the old version in the live environment, but only direct internal traffic to the new version. This allows full-scale testing without affecting end users.
- **Recreate Deployment:** Temporarily shut down the old version of the application, then bring up the new version. This is simpler but incurs downtime.

Monitoring and Post-Deployment

- **Real-time Monitoring:** Utilize tools like Prometheus and Grafana for continuous system performance tracking.
- **Alerts and Notifications:** Automate alerts via platforms like Slack or email for immediate issue notification.
- **Log Management:** Employ ELK Stack or Splunk for efficient log analysis and problem diagnosis.
- **Performance Metrics:** Use APM tools to monitor crucial metrics such as load times and error rates.
- **Continuous Feedback Loop:** Establish feedback mechanisms to continuously improve the system based on user data and system performance.

How to organize CI/CD better?

- Continuous Integration and Continuous Delivery are the best practises as they create an effective process of integrating and delivering the product to the market
- Small code changes can be made in the software code which makes the entire process to be simpler and easier
- CI and CD provides continuous feedback from the customers and also from the DevOps team. This increases the transparency of any problem in the team or outside
- The process enables faster release of the product. The failures can now be detected faster and hence fixed faster, which increases the speed of release

Security in CI/CD

- Integrate automated security scans with tools like SonarQube and OWASP ZAP to analyze code for vulnerabilities. Manage secrets securely using Vault by HashiCorp or AWS Secrets Manager to prevent exposure in the codebase. Enforce code review practices and automated checks to uphold secure coding standards. Utilize policy-as-code frameworks such as Open Policy Agent to automatically enforce security policies. Implement continuous security monitoring to detect and respond to threats in real-time.

Scalability and Efficiency

- **Containerization and Orchestration:** Deploy Docker for containerization to encapsulate applications in portable environments. Use Kubernetes to automatically scale these containers up or down based on traffic and load, optimizing resource usage.
- **Microservices Architecture:** Break down the application into smaller, independent services that can be scaled individually. This approach allows for targeted scaling and updates, reducing downtime and improving response times.
- **Pipeline Optimization:** Parallelize tasks within the CI/CD pipeline to reduce build and deployment times. Implement caching strategies to reuse previously fetched or compiled data, significantly speeding up the process.
- **Performance Monitoring:** Regularly monitor metrics like CPU usage, memory consumption, and response times. Use this data to adjust resources dynamically, ensuring efficient operation without overprovisioning.
- **Infrastructure as Code (IaC):** Leverage tools like Terraform to manage infrastructure provisioning and deployment through code. This enables quick replication of tested environments, reduces human error, and enhances consistency across deployments.



Future Trends in CI/CD

AI and Machine Learning in CI/CD

- Use machine learning models to predict and prevent deployment failures by analyzing historical data. Integrate AI to automate and optimize test case generation and analysis, improving testing efficiency and coverage. Enhance monitoring systems with AI for automated anomaly detection, allowing for faster responses to performance issues. Apply machine learning algorithms to optimize resource allocation and usage, ensuring cost-effective performance. Implement continuous learning mechanisms to allow AI systems to evolve and adapt processes based on operational feedback.

Infrastructure as Code

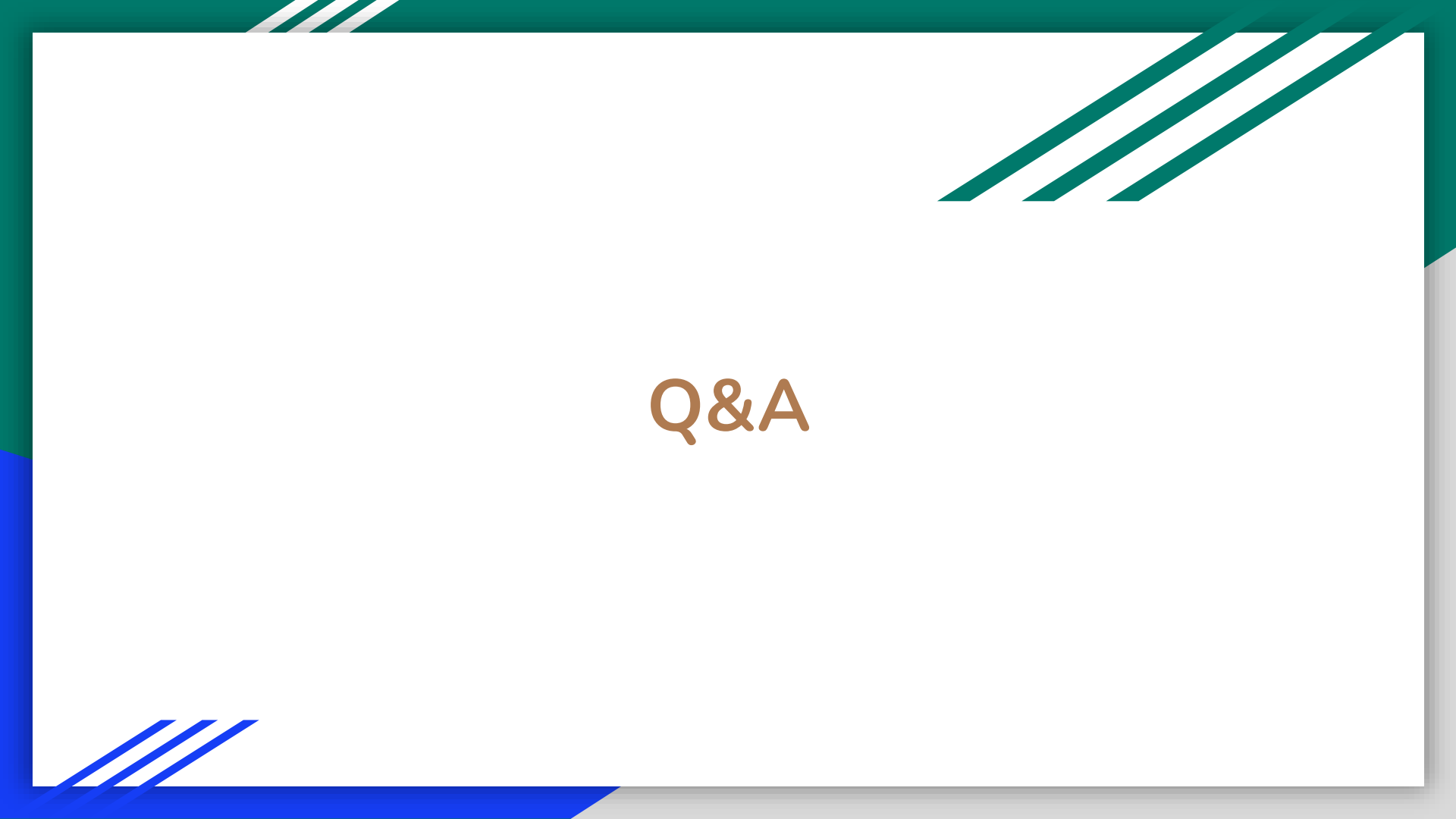
- Infrastructure as Code (IaC) uses code to manage and provision infrastructure, automating the setup of servers, databases, and other network components. This practice ensures consistency and speed in building and scaling environments. Tools like Terraform, Ansible, and AWS CloudFormation allow developers to define infrastructure requirements using code, which can be version-controlled and reused, minimizing human errors and streamlining deployment processes. IaC supports rapid iteration and reproducibility across development, testing, and production environments, facilitating efficient management and quick adjustments in response to changing needs.

The Role of Cloud Services

- Cloud services play a pivotal role in CI/CD pipelines by providing scalable and flexible resources that can be dynamically adjusted to meet the demands of development and deployment processes. They offer key services like cloud-based build servers, storage, and environments that facilitate faster and more efficient pipeline execution. Platforms such as AWS, Azure, and Google Cloud provide tools that integrate directly into CI/CD workflows, supporting activities like automated testing, deployment, and monitoring. Cloud services enable organizations to leverage advanced technologies without the upfront cost of physical infrastructure, reducing overhead and enhancing agility.

DevSecOps Integration

- DevSecOps integrates security practices within the CI/CD pipeline, embedding security at every phase of software development and operations. This approach ensures that security considerations are not an afterthought but a fundamental aspect of the development process. Key practices include automated security scanning, continuous compliance monitoring, and the use of security-as-code techniques to enforce security policies programmatically. DevSecOps fosters a culture where development, operations, and security teams collaborate closely, leading to faster detection and remediation of vulnerabilities, ultimately enhancing the security posture of applications.



Q&A